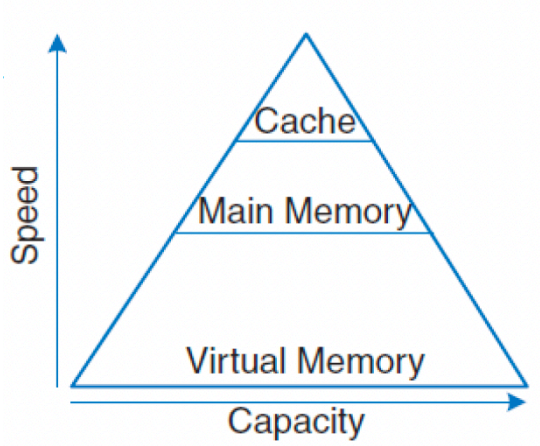
Memory Hierarchy

Description

Cost vs Capacity vs Access time

- **SRAM**
  - Access Time (ns): 0.3 – 0.8 Bandwidth (GB/s): 200+
  - Price ($/GB): 4,500
  - Used for: registers and **caches**
- **DRAM**
  - Access Time (ns): 8 - 40 Bandwidth (GB/s): 25-50
  - Price ($/GB): 6
  - Used for: **RAM**
- **Flash memory**
  - Access Time (ns): 18,000 (18 us) Bandwidth (GB/s): 0.6 (10 for PCIe 4)
  - Price ($/GB): 0.35
  - Used for: SSD disk (secondary and **virtual memory** – non volatile)
- **Magnetic disks**
  - Access Time (ns): 4,500,000 (4.5 ms) Bandwidth (GB/s): 0.8
  - Price ($/GB): 0.04
  - Used for: HDD disk (secondary/tertiary storage – non volatile)



NVRAM technology

NVDIMM: A non-volatile memory technology that fits in standard DIMM slots, providing **byte-addressable** access similar to DRAM but with persistent storage capabilities.

NVMe: A high-speed protocol leveraging PCIe for block-level access, optimized for non-volatile memory (e.g., SSDs). Designed for storage

Device	Latency (ns) (read/write)	Bandwidth (GB/s) (read/write)	Price ($/GB)
DDR4 DRAM	100/100	100/37	6
NVDIMM (Optane)	370/400	38/10	20-30
PCIe NVMe SSD	10000 (10 μs)	7/5	0.3-0.4

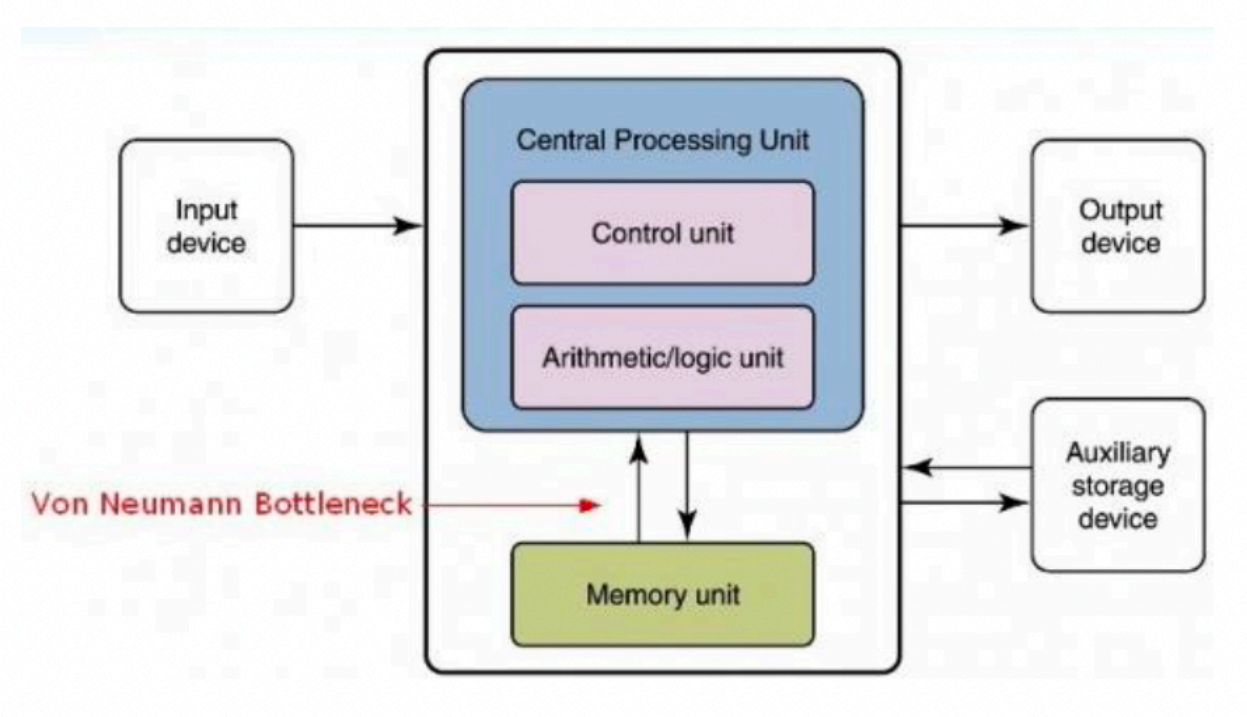
💡 Larger memory is slower and cheaper

💡 Smaller memory in faster and more costly

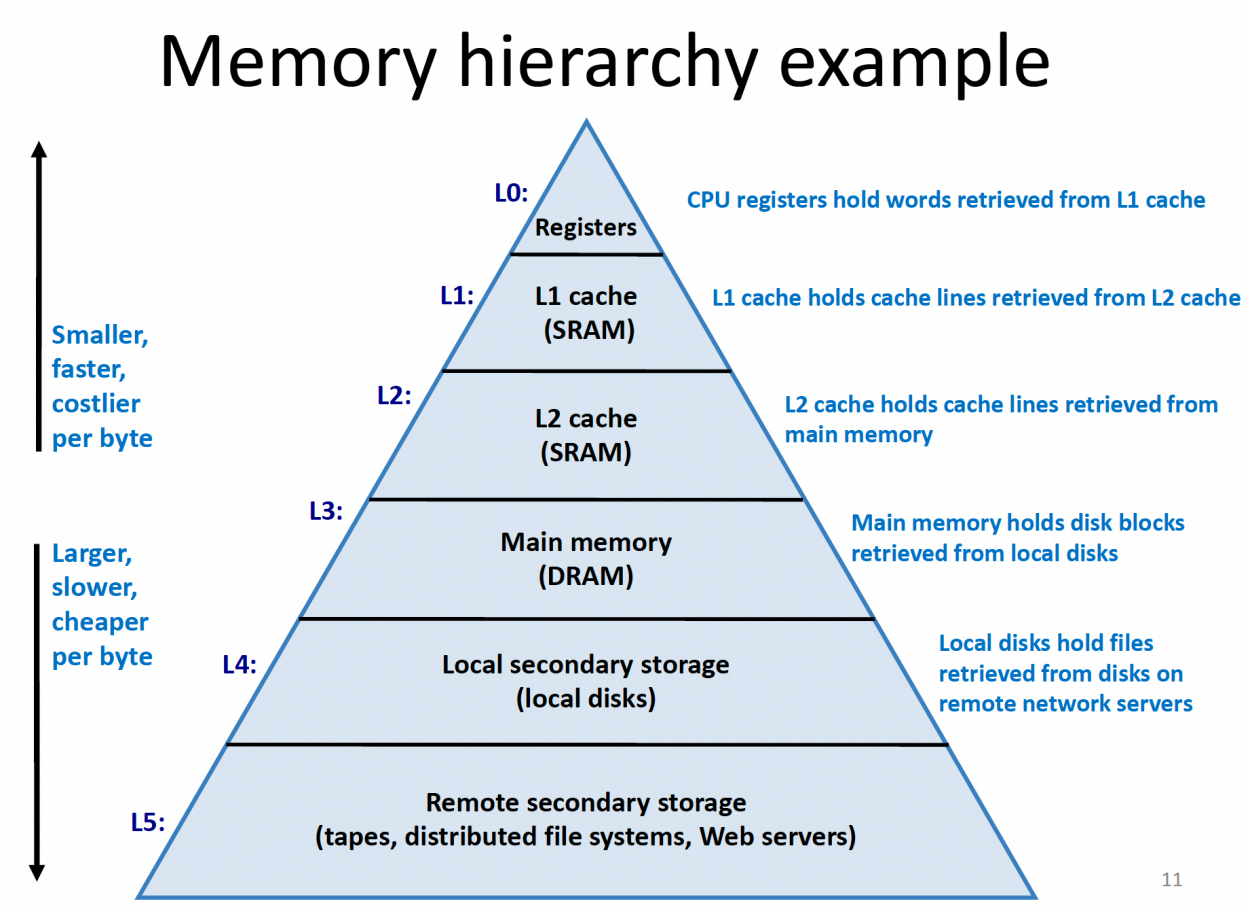
Vn Nerymann Architecture

The von Neumann architecture separates the processing unit from the memory unit.
CPU and Memory share a bus for instruction fetch and data access.
This design introduces the so-called **von Neumann bottleneck**, which limits data transfer rates between the CPU and memory.

In early computer systems timings for accessing main memory and for computation were reasonably well balanced.



The performance of von Newmann architectures is constrained by the speed disparity between
CPUs and the maximum data transfer rates from off-chip memory to on-chip memory (e.g., registers).



Terminoly

Hit: If the data the processor requests is present in one block at the closest memory level

Hit rate: is the fraction of memory accesses found in the upper level

Miss rate: fraction of memory accesses not found in the upper level

Miss penalty: the time to replace a block in level n with the corresponding block from level n-1

Miss time: the time to retrieve an element in case of a miss

> 💡 ***miss time = miss penalty + hit time***

Miss rate

$$MR = \frac{number\ of\ misses}{number\ of\ total\ memory\ accesses} = 1 - HR$$

Hit rate

$$HR = \frac{number\ of\ hits}{number\ of\ total\ memory\ accesses} = 1 - MR$$

The **Hit Rate** is the *probability* to have an hit in the upper memory level when executing a memory access instruction (load or store).

> 💡 *If we have a miss (with probability 1- HR), it means that the memory access instruction takes more time to be completed*

AMAT (Average Memory Access Time

*The **Average Memory Access Time (AMAT)** is defined as:*

$$hit\ time\ + (1 - hit\ rate) * miss\ penalty$$

MR_x iss the Miss Rate (probability of miss) at memory level x

t_x is the access time for a memory hit at level x (*hit time*)

If we have multiple memory hierarchy levels, then AMAT is given as follows

$$AMAT = \underbrace{t_{M1}}_{\text{hit time}} + \underbrace{MR_{M1}}_{\text{miss rate}} * \underbrace{(t_{M2} + MR_{M2} * (t_{M3} + MR_{M3} * (t_{M4} + \ldots)))}_{\text{miss penalty}}$$

> 💡 If the HR is high enough, the AMAT approaches that of the highest (and fastest) level. In addition, there is the illusion that the memory size is equal to that of the lowest (and largest) level.

The locality principle

To achieve high hit rates in a memory hierarchy, system designers and programmers rely on the **principle of locality**, which describes patterns in how programs access memory.

Locality of reference (or ***locality principle***) refers to the phenomena in which a program tends to access *the same set of memory locations over a given period of time*.

It has been observed that If the program refers to a memory location, then:

1. *The same memory location* will be referenced again soon with a high probability (**temporal locality or data reuse**)
2. The memory locations "*close to*" the memory location just accessed will be referenced soon with a high probability (**spatial locality**)

> 💡 The *locality principle* is the ***driving force*** that ensures the memory hierarchy works properly

It increases the probability of reusing data blocks that were previously moved from level n to level $n - 1$ (i.e., closer to the CPU), thus increasing the *hit rate* (reducing the *miss rate*).

> 💡 *Locality is crucial for optimizing memory hierarchy performance.*

Locality characterization

Temporal locality (or ***data reuse***): data items referenced recently are likely to be referenced again soon

Spatial locality: items near to those referenced recently are likely to be referenced again soon

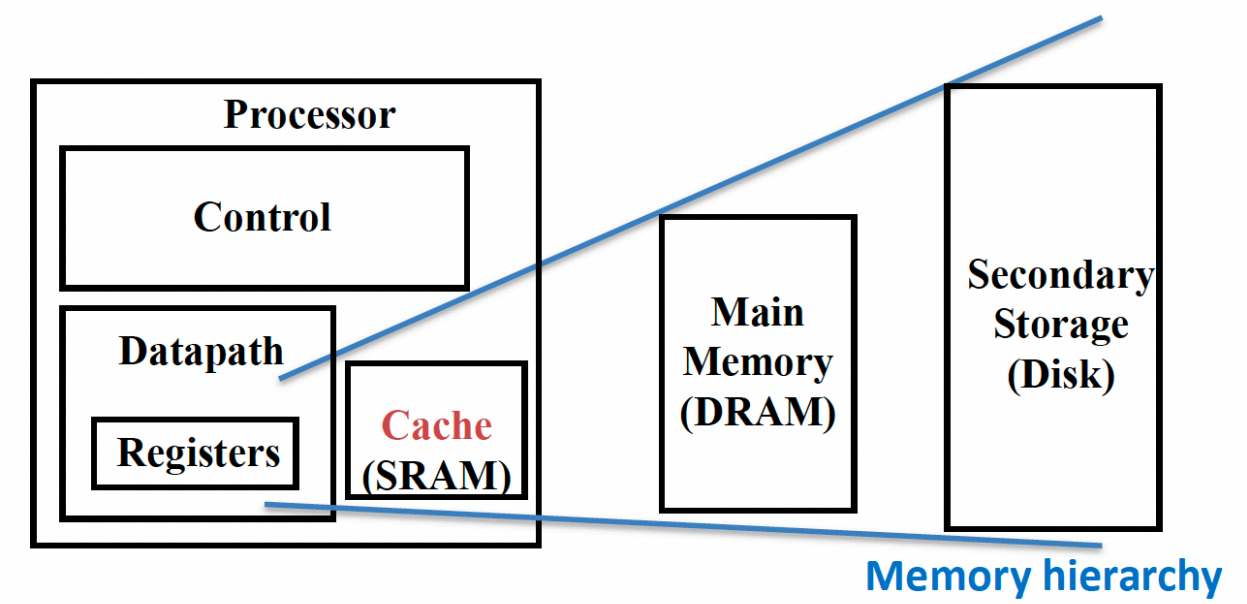
Data transfering

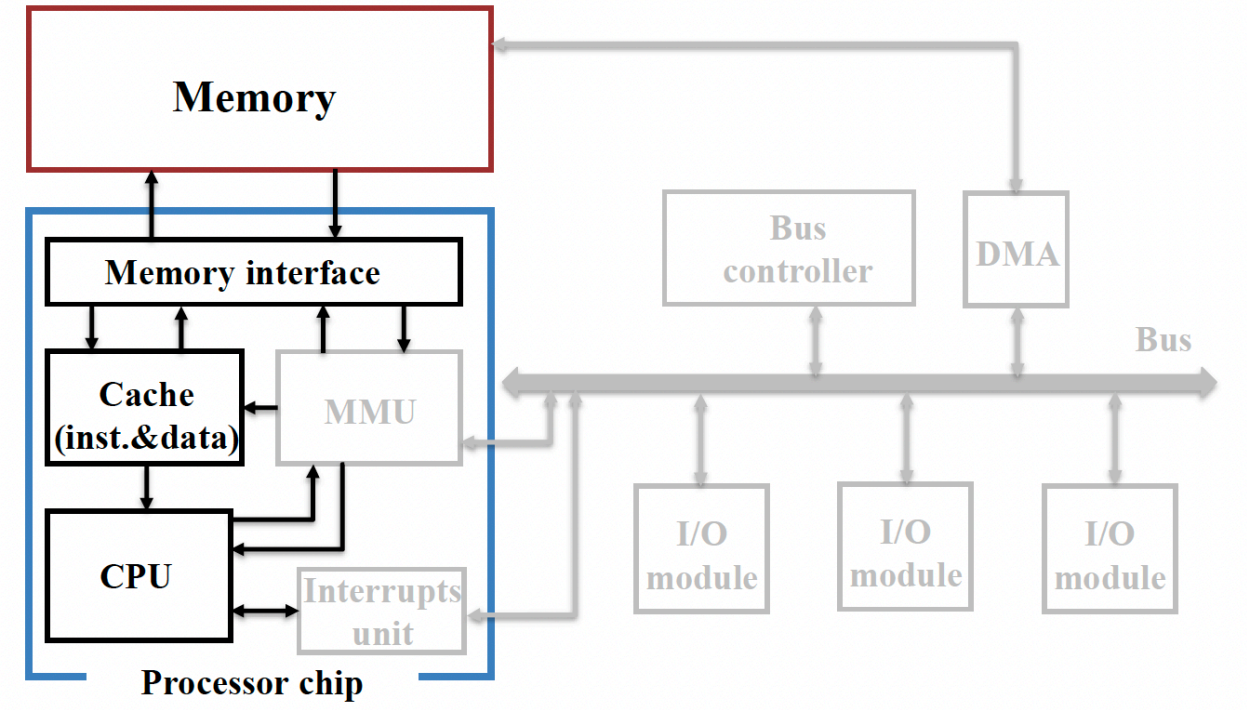
Data is transferred between only two adjacent memory levels at a time

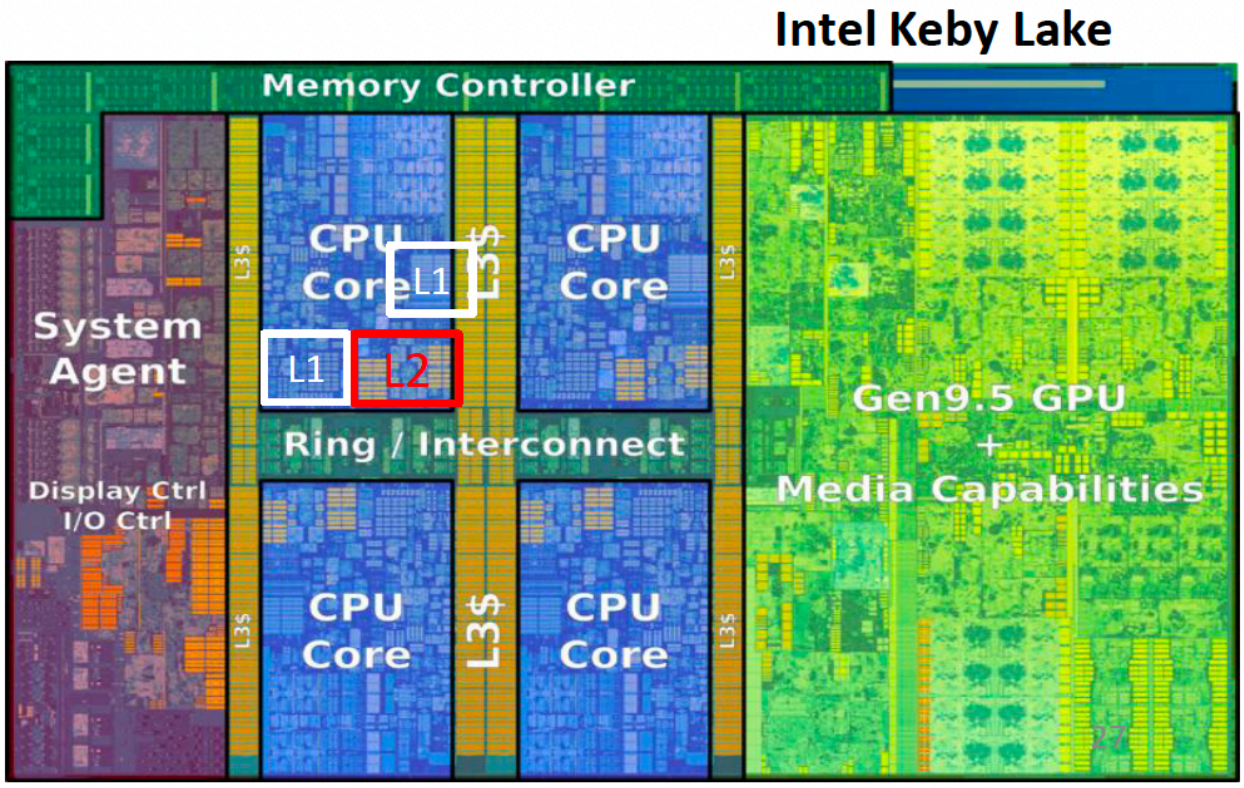
Transfers always happen at ***block*** granularity between memory levels to exploit spatial locality

Cache memories

On modern architectures, the **cache** memory is integrated on-chip, representing the fastest levels of the memory hierarchy.







Cache usage

Caches are organized in ***lines***, *each line contains a block of memory words (e.g., 8 - 16 memory words).*

The first time the processor asks for a memory word a ***cache miss*** occurs

Cache Hit: If the data is present in a cache line
Cache Miss: If the data is not present (the block containing the data is transferred into a cache line)

Cache effects on AMAT

The usage of large caches in a memory hierarchy helps to reduce the Von Neumann Bottleneck

Quantitative example. Suppose the following case (2 levels):

- $t_M = 50ns$ (*main memory service time*)
- $t_{L1} = 1ns$ (L1 hit time, i.e., cache hit service time)
- Miss rates (MR_{L1}): 5%

1. No cache: *AMAT*= **50ns**
2. With L1 cache:
   AMAT= $t_{L1} + MR_{L1}$* t_M = 1+0.05*50= **3.5ns**

We have an improvement of more than 14X!

Cache miss rates

SPECCPU2006 benchmarks

Intel Core i7 920 @2.66GHz

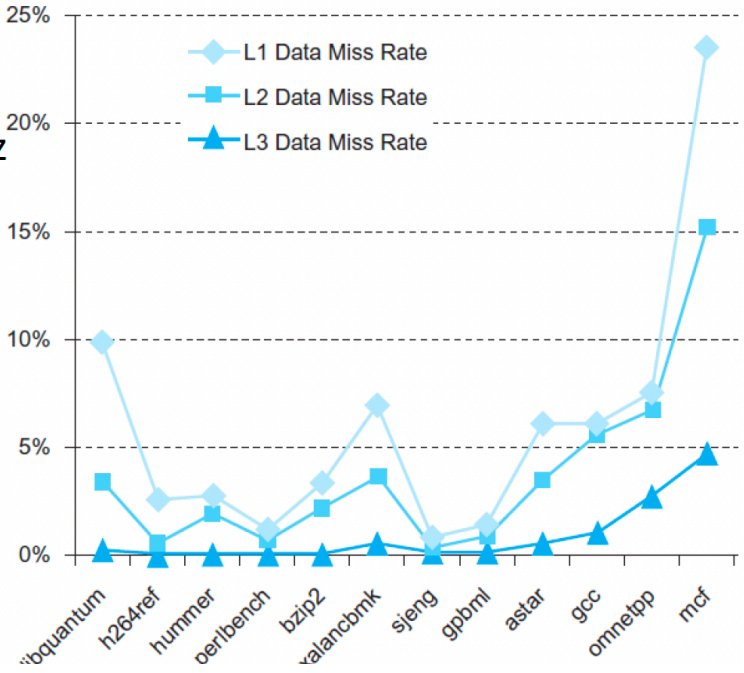
- 4 cores, 8 HW contexts
- L1d 4x32KB, L1i 4x32KB
- L2 4x256KB
- L3 8MB shared

t_M about 100 cycles

t_{L1} 4 cycles (~1.5ns)

t_{L2} 10 cycles (~3.7ns)

t_{L3} 35 cycles (~13.16ns)



CPU time with caches

$$CPU_{time} = ClockCycles * ClockCycleTime = IC * CPI * ClockCycleTime$$

- IC (Instruction Count) is the number of program instructions executed
- $IC = IC_{CPU} + IC_{MEM}$
- CPI (ClockCycles per Instruction) is defined as $\frac{ClcockCycles}{IC}$

CPU time can be further divided into the cycles that the CPU spends executing the instructions **with no misses** ($\boldsymbol{CPI_{perfect}}$) and the clock cycles that the CPU spends **waiting for the memory system** ($\boldsymbol{CPI_{stall}}$ *or Memory-stall ClockCycles*)

$$CPU_{time} = (CPI_{perfect} + CPI_{stall}) * ClockCycleTime$$

$$\text{CPI} = \left(\frac{IC_{CPU}}{IC}\right) * CPI_{CPU} + \left(\frac{IC_{MEM}}{IC}\right) * CPI_{MEM}$$

Considering that each memory instruction may generate a cache hit or miss

$$CPI_{MEM} = CPI_{MEM-HIT} + Miss\ rate * CPI_{MEM-MISS}$$

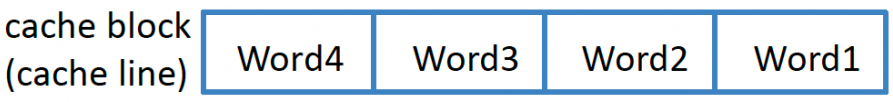
$$\text{CPI} = \frac{IC_{CPU}}{IC} * CPI_{CPU} + \frac{IC_{MEM}}{IC} * CPI_{MEM-HIT} + \underbrace{\frac{IC_{MEM}}{IC} * Miss\ rate}_{\text{Miss rate per memory instruction}} * Miss\ penalty$$

$CPI_{Perfect}$

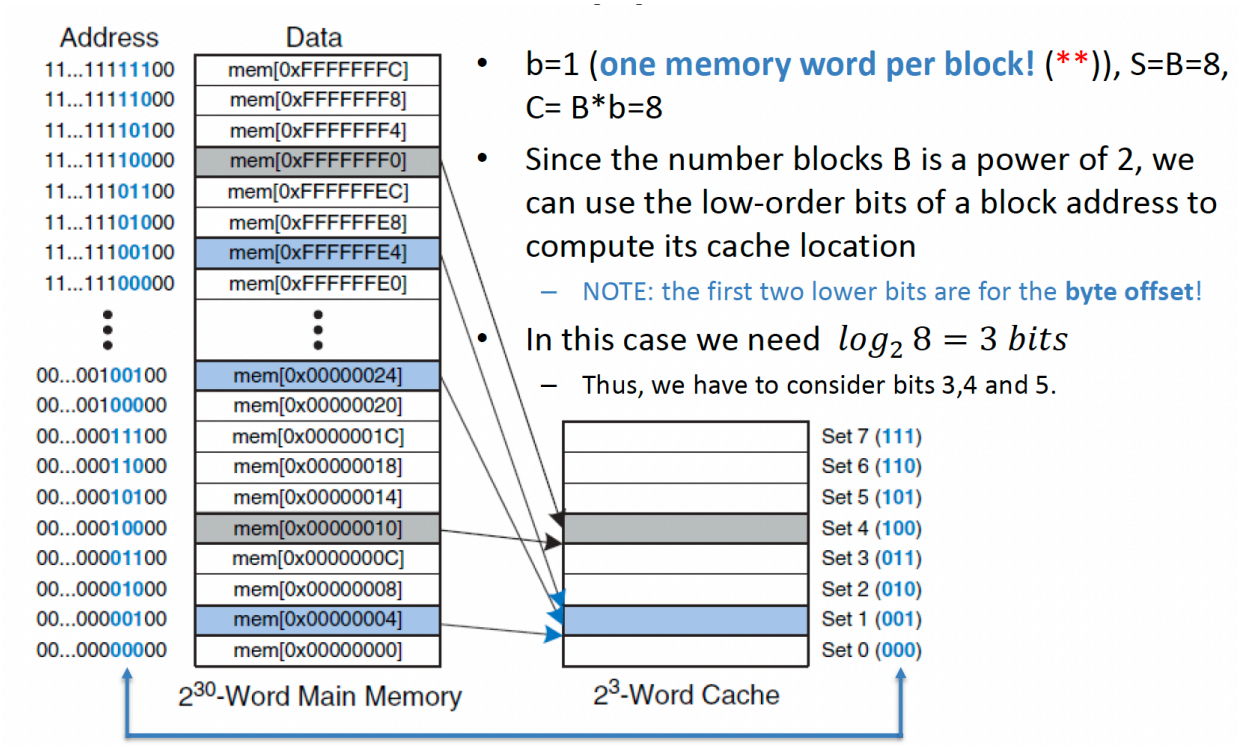
CPI_{Stall}

Organization of caches

- **Direct mapped**: Cache ha #S=#B
- **N-way set-associative** cache. Each set contains B blocks. $S = \frac{B}{N}$
- **Fully associative** cache has S=1



Direct mapped cache



- b=1 (**one memory word per block!** (**)), S=B=8, C= B*b=8
- Since the number blocks B is a power of 2, we can use the low-order bits of a block address to compute its cache location
  - NOTE: the first two lower bits are for the **byte offset**!
- In this case we need $log_2 8 = 3\ bits$
  - Thus, we have to consider bits 3,4 and 5.

How do we know which block is in a cache location?

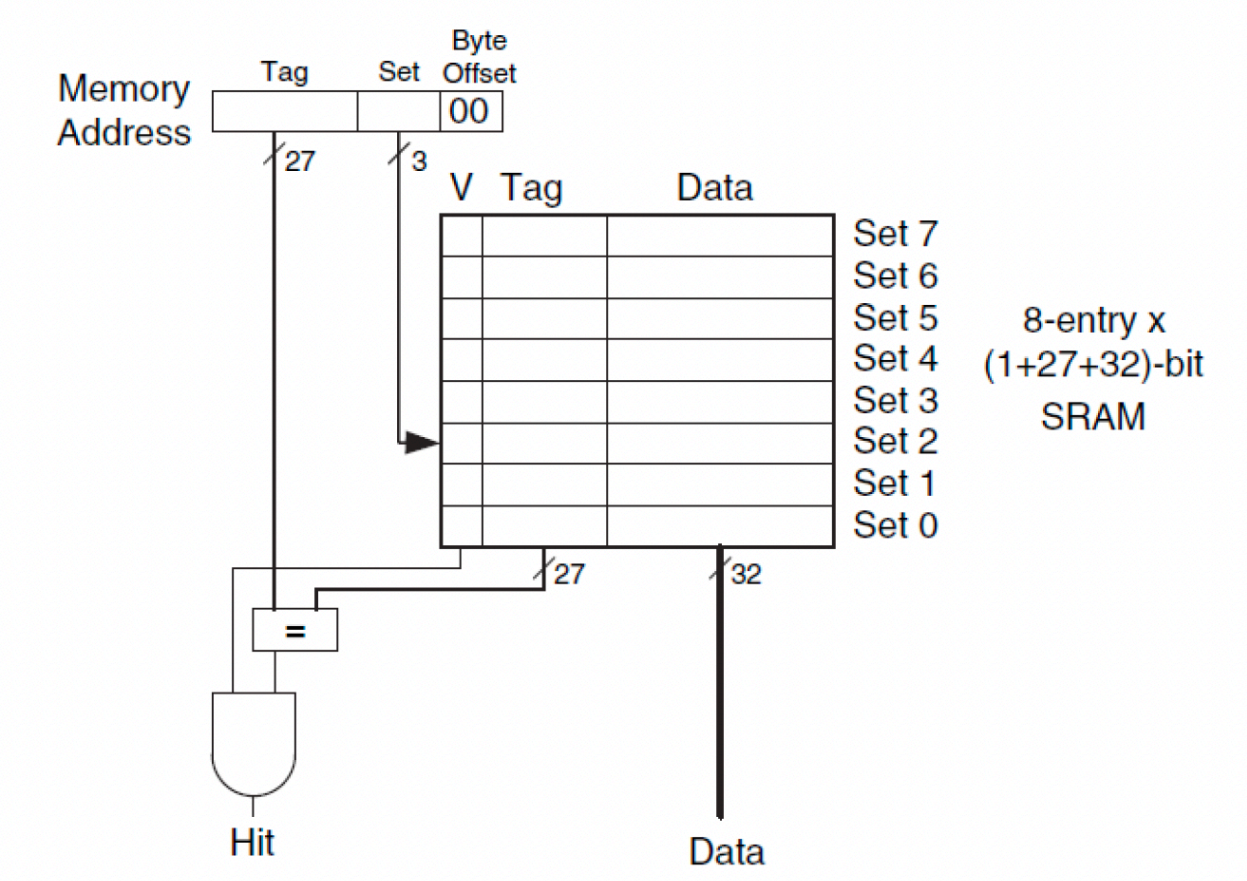
We need a set of **tags for each cache location** identifying the block address in the cache, actually, the tag only needs to contain the higher-order bits of the memory address



How do we know whether the data in a cache location is valid?

We need **a valid bit to each cache location** (`valid=1` ; `invalid=0`)

Direct mapped cache



Pros:

- simple to realize
- very fast in case of cache hits (1 clock cycle)

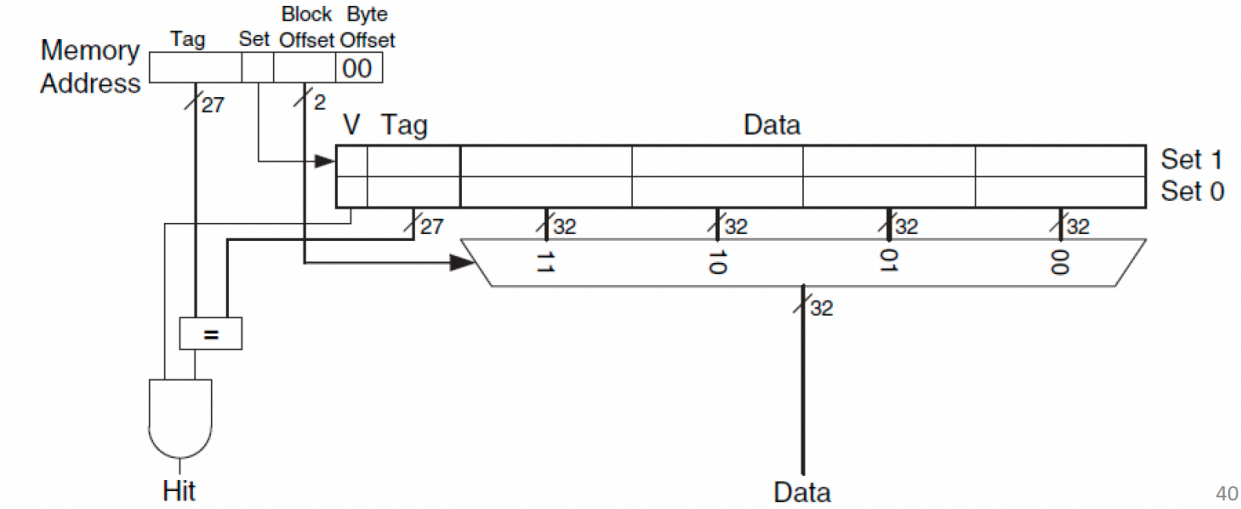
Cons:

- Too rigid, a given memory word can be placed in only one single entry of the cache (direct mapping between memory addresses and cache lines)
- This rigidity *may have a significant impact on the number of cache conflicts* (***thrashing***) depending on the memory placement and usage of the data structures
- The substitution of a cache line does not consider *temporal locality*

Cache thrashing occurs when frequent evictions and reloads of data happen despite free space being available in the cache.

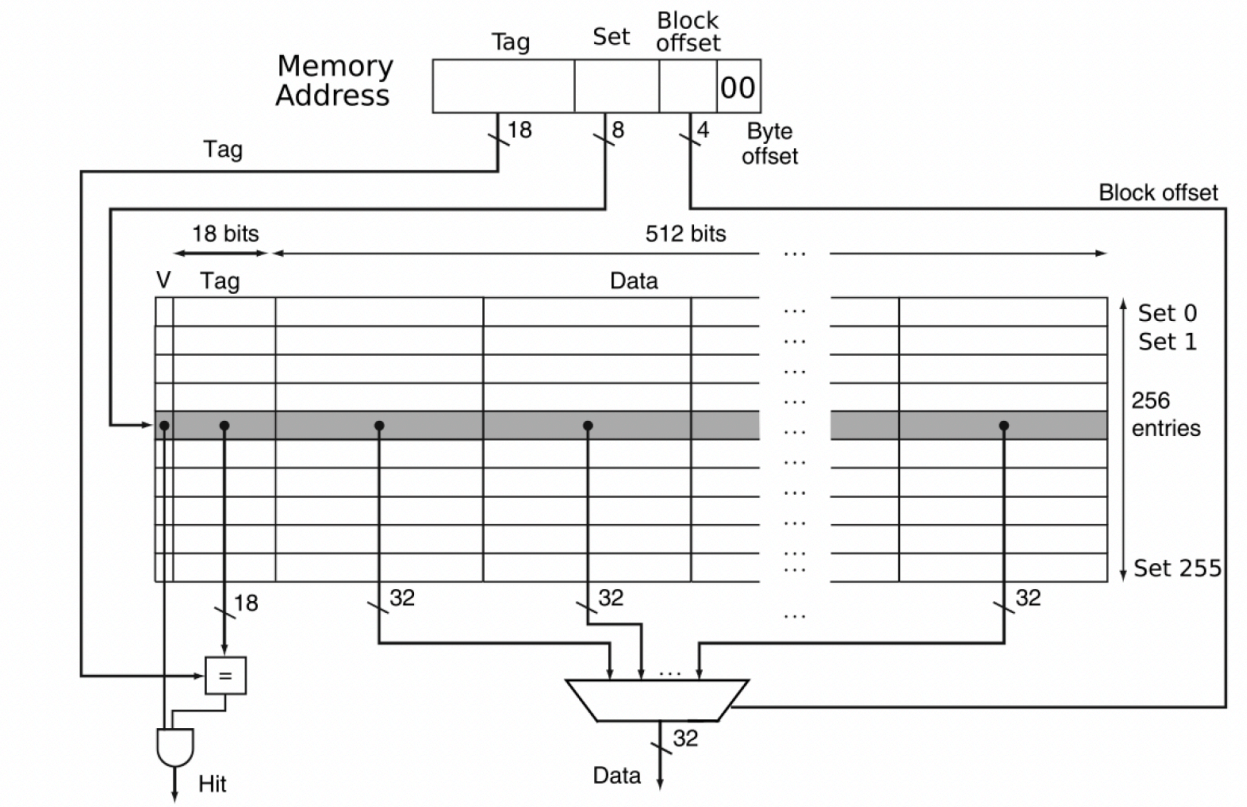
Direct mapped cache with `b > 1`

To take advantage of ***spatial locality***, a cache uses larger blocks containing several consecutive words (typically 8 or 16).



40

256x64 Byte Blocks Direct Mapped Cache



Associative Caches

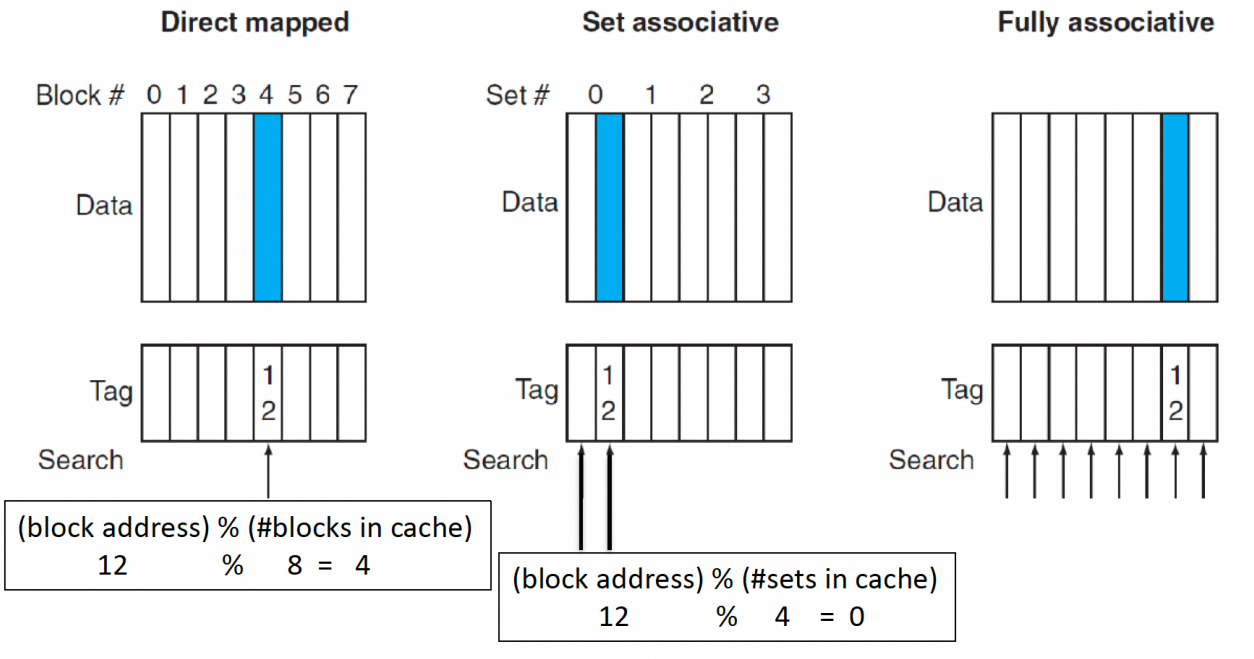
We can use associative caches **to reduce conflicts**

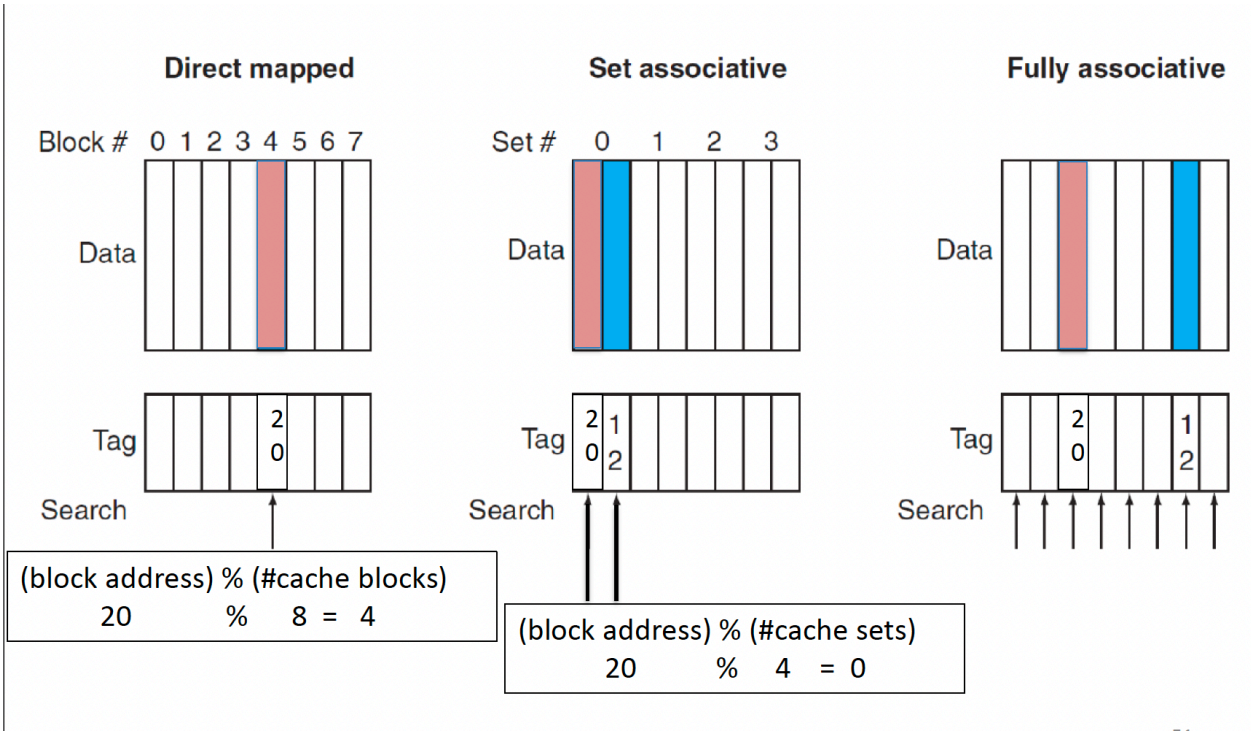
Fully associative cache

- Blocks of memory words can be placed in any cache line in the cache
- To find a given block it requires searching in all entries (**in parallel !!**)
- Each entry has a comparator. Thus, the HW cost significantly increases

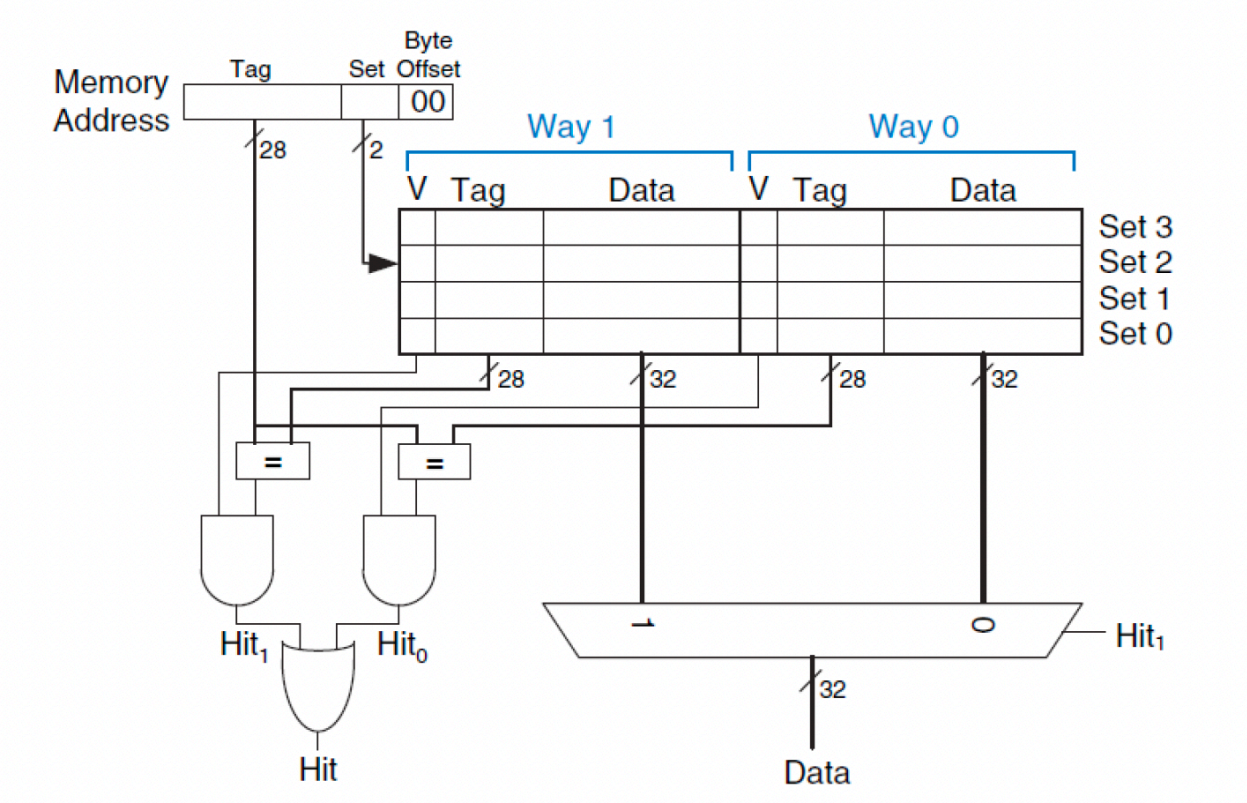
N-way set-associative cache

- Each block address is mapped *to exactly one set* (as in the direct mapped cache where #S=#B), but each set contains N cache lines
- A block can be placed in any cache line of the set (as in the fully associative cache, i.e., it is fully associative if we consider one Set)
- To find a given block, the search is made in parallel in the N entries of the set using N distinct comparators and reducing the cost compared to fully associative caches

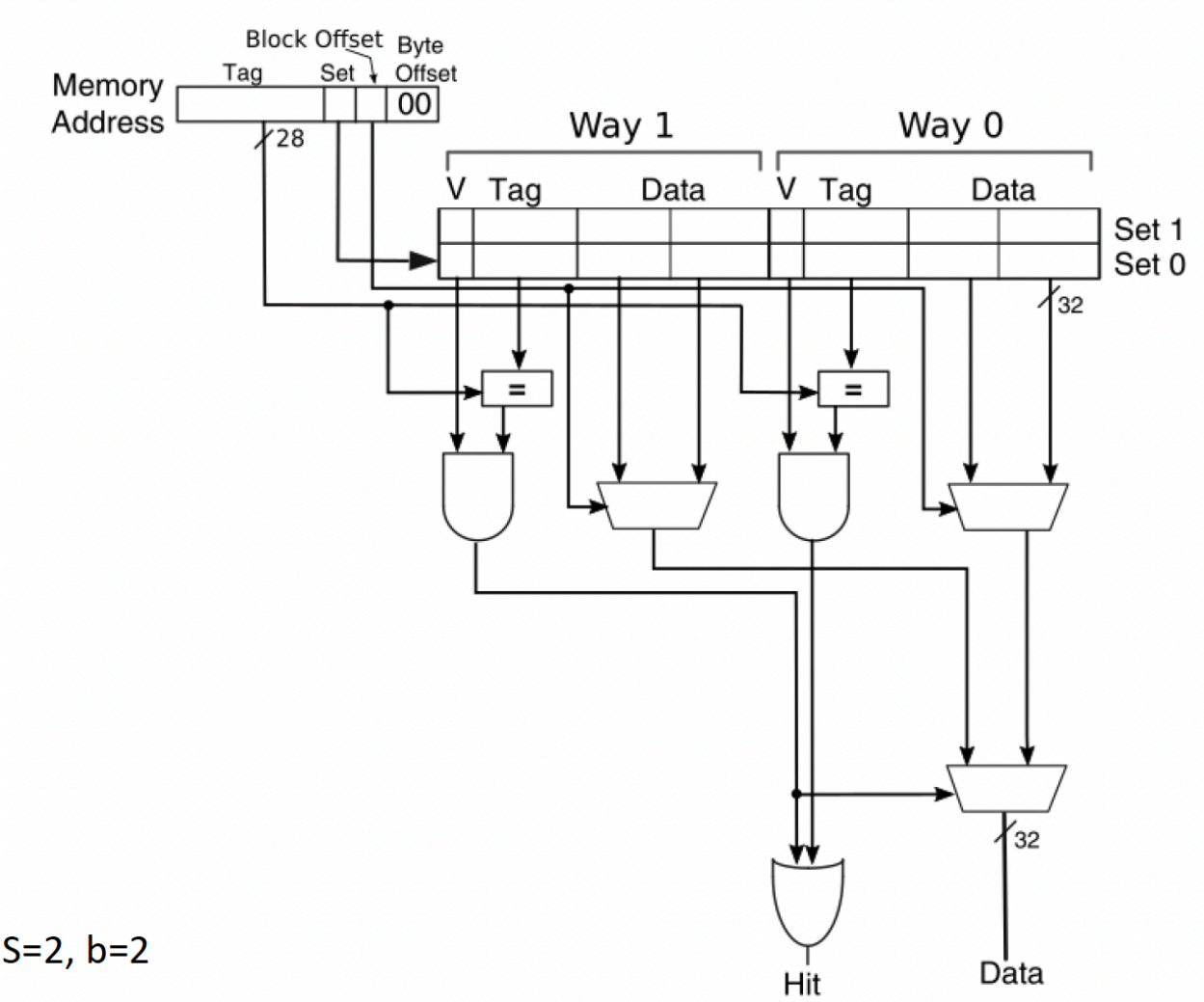




2-way Set-Associative Cache (b=1)

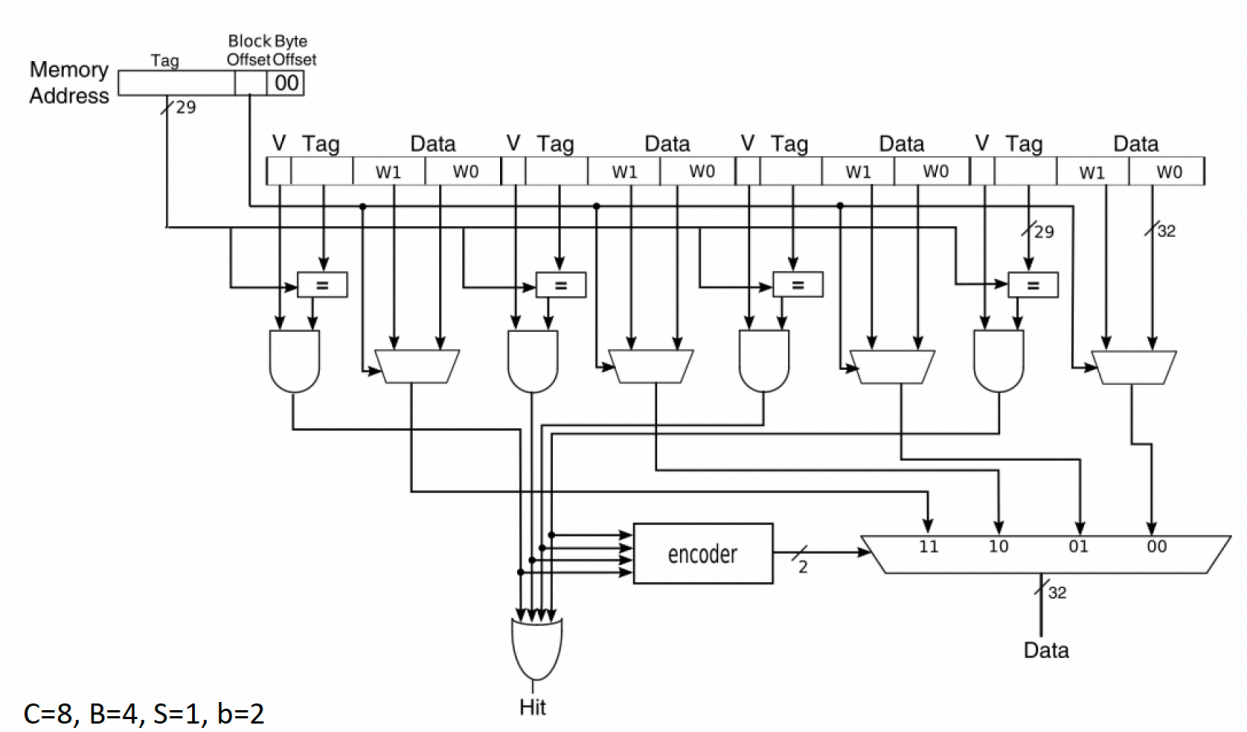


2-way Set-Associative Cache (b=2)



S=2, b=2

Fully Associative Cache (b=2)



C=8, B=4, S=1, b=2

Cache organization summary

Cache misses can be reduced by increasing the capacity C, the block size b and the associativity N

Organization	Number of Ways (N)	Number of Sets (S)
Direct Mapped	1	B
Set Associative	$1 < N < B$	B/N
Fully Associative	B	1

Cache Working Set

The working set (WS) of a program for a given memory hierarchy is defined as the set of data actively accessed by the program during a specific time interval

Handling cache misses

A cache miss *stalls the entire processor*, freezing the contents of all registers while waiting for memory,

Indeed, modern CPUs employ advanced techniques to mitigate such stalls:

- *out-of-order execution* of other instructions while waiting for memory
- *speculative execution* of branches
- *prefetching* of future memory accesses preloading data into the cache
- SMT (Simultaneous Multithreading) to execute instructions of other threads

The steps which are taken for a *cache miss* (i.e., ***fault management***):

1. Tell the next memory level to *read* the missing value
2. Wait for the memory to respond (this can take multiple cycles)
3. Update the corresponding cache line with the data received
4. Restart the instruction execution (now it is a cache hit)

Kinds of cache misses

Compulsory misses

These are cache misses caused by the first access to a block that has never been in the cache. These misses are **unavoidable.** Can be mitigate by using prefetching and large b

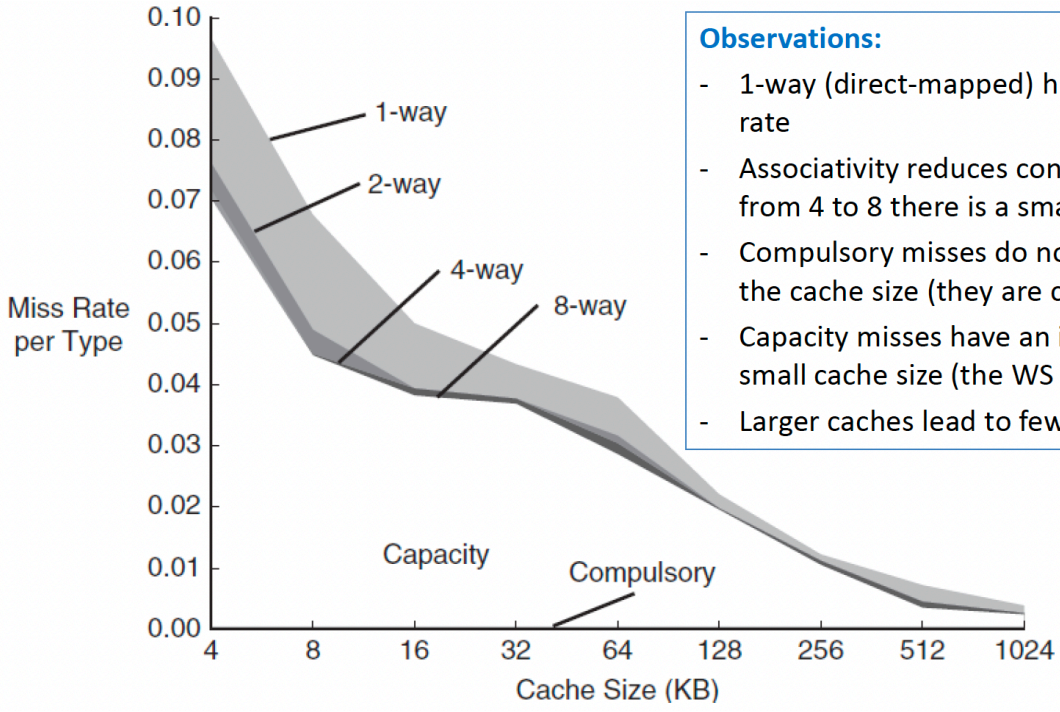
Capacity misses

These are cache misses caused when the cache cannot fit the WS (i.e., the WS is larger than C). Some blocks are evicted and later retrieved. Can be mitigated by increasing the cache size and through optimized algorithm that reduces the WS by changing the access pattern (e.g., blocking, tiling)

Conflict misses

Only for direct mapped and set-associative caches. These are cache misses that are *eliminated* in a fully associative cache of the same size.

Can be mitigated by improving data placements or through the so-called ***victim caches***, i.e., small fully associative caches used as buffers for evicted blocks (typically from L1)



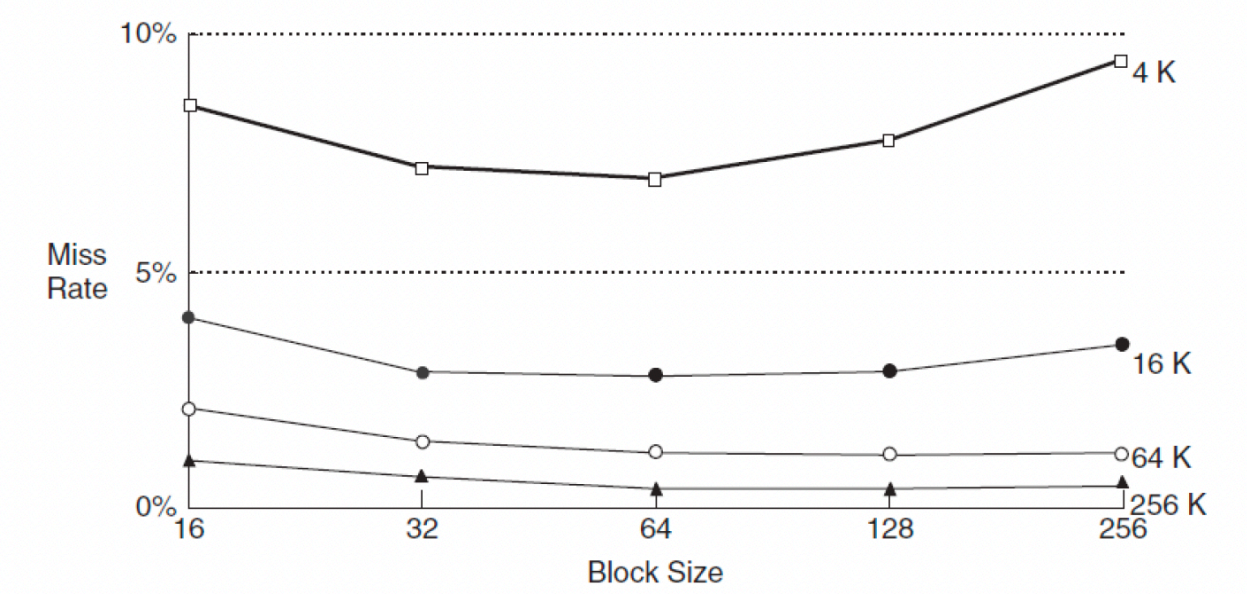
Observations:

- 1-way (direct-mapped) highest miss rate
- Associativity reduces conflict misses; from 4 to 8 there is a small difference
- Compulsory misses do not depend on the cache size (they are constant)
- Capacity misses have an impact for small cache size (the WS does not fit)
- Larger caches lead to fewer misses

Miss rate vs block size and cache size

As block size increases, there are more opportunities to exploit spatial locality, but:

1. Increases the probability of conflict misses because the number of sets decreases (in each curve C is fixed!)
2. The miss penalty increases (more time to load a block)



Multi-level caches in modern processors

💡 Using the multi-level caches helps to reduce the miss-penalty

If the L2 cache contains the desired data, the miss penalty for the L1 cache will be essentially the access time of the L2 cache, which will be much less than the access time of the main memory

💡 With **multilevel caches**, the memory-stall clock cycles can be defined as the sum of the stall cycles coming from all cache levels

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

Write-Through (WT)

Write hits always update both the cache and the next memory level

Pros:

- simple solution, easy to implement
- Data is ***always coherent*** between the two consecutive memory levels

Cons:

- The speed of writes depends on the write speed of the lower memory level
- Higher memory traffic: for every single write, there may be multiple writes in each memory level

Write-Back (WB)

Write hits update only the cache, then the modified block is written to the lower memory level when it is replaced

Pros:

- The speed of writes is that of the cache
- Lower memory traffic than WT policy. Subsequent writes to the same cache block do not produce traffic with lower memory levels

Cons:

- We need to keep track of modified blocks (we need an extra bit: **dirty bit**)
- More complex to implement than WT
- The cache line replacement is more costly because it may require to transfer the data block to the lower memory level (**if the dirty bit is 1**)

However, the cost of cache writes is higher than WT if a write miss occurs because we first must write the block back to memory (*only if the dirty bit is 1*) and then execute the write.

However, we may use a **write buffer** also for WB to temporarily hold the data to write while the cache block is checked for a hit.

Handling write misses

Write misses (different TAG)

Two policies: 1) write-allocate; 2) no-write-allocate

Write-Allocate (WA)

It fetches the block from memory to cache and then overwrite the word into the cache block. The block is loaded into the cache followed by a write hit. Mainly used in the Write-Back policy. Higher latency but optimizes for temporal locality (assumes data will be accessed again soon)

No-Write-Allocate (NWA)

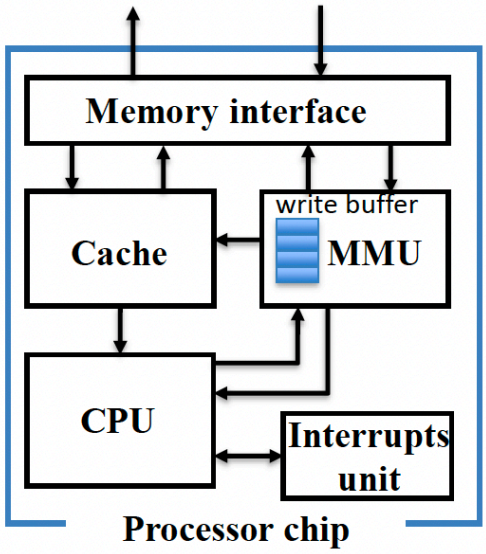
It writes the word directly to the next memory level without fetching the block into the cacheThe block is not loaded into the cache. Mainly used in the Write-Through

policy.
Lower latency than WA, but does not optimize potential data reuse

Optimizing writes

A **write buffer** is used to temporarily hold the data waiting to be written to memory. Execution continues immediately after writing the data into the cache **and** into the write buffer

💡 The CPU stalls only if the write buffer is full



💡 *For WT caches, the write buffer is mandatory, otherwise, the performance is too low.*

💡 *For WB caches, it is an optimization adopted in most of the cases.*

Write-Back with Write Buffer

The write buffer in WB is used to:reduce the Miss Penalty during block replacement. Reduce latency during cache operations

- **Write Hit**:
  the processor does the cache lookup **and** places the data in the write buffer during the normal cache access cycle. The new data is written from the write buffer into the cache on the next unused cache access cycle (***pipelining***)
- **Write Miss**:
  We cannot overwrite the block in the WB cache! If the dirty bit is 1, the modified block is moved into the write buffer while the new block is fetched from memory. When the new block arrives, the new data is written into the cache (write allocate policy).

Cache replacement

In a *direct mapped* cache, the requested block can go in exactly one position, thus we have no choice

> 💡 If the block to be replaced has been modified and the write policy is Write-Back, we must update the lower-level memory

In an associative cache, we have flexibility in choosing where to place the requested block

Cache replacement policy

When a cache set is full, a replacement policy determines which block to evict to make room for a new block.

The most used scheme is Least Recently Used (LRU). It leverages temporal locality, the block replaced is the one that has been ***unused*** for the longest time. LRU ensures that recently used blocks are terained

Implementation: For a 2-way set-associative cache, it can be implemented with 1 bit (*use bit -- U*), for a 4-way is still feasible with 2 bits. For more than 4-ways it becomes increasingly complex, thus *pseudo-LRU* is often used

For high associative caches, a **random** policy provides approximately the same performance as LRU with reduced complexity.

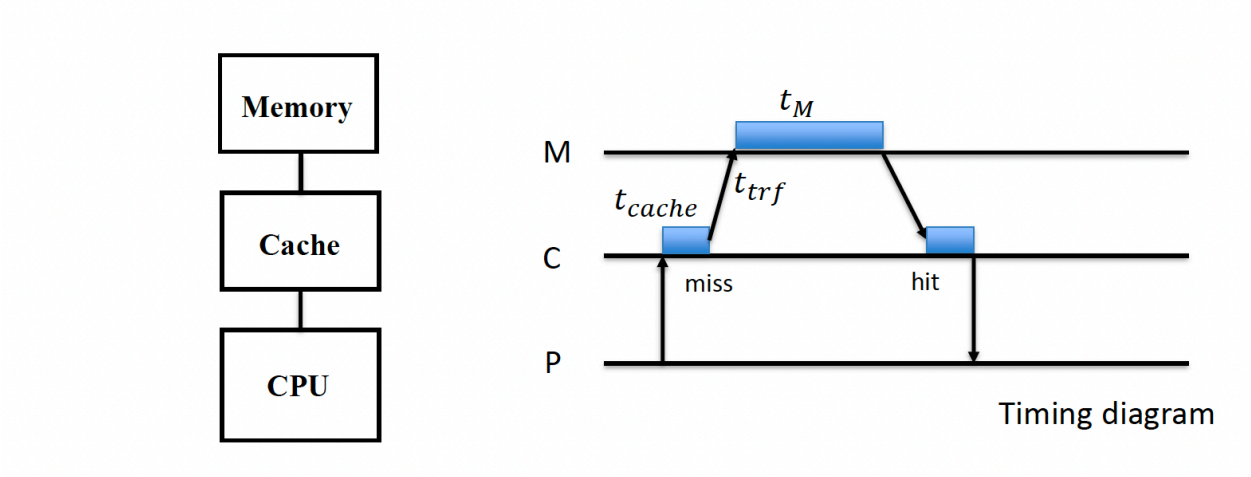
Designing the memory system

The *miss penalty* can be reduced improving memory organization thus increasing the bus bandwidth between DRAM and cache

Three possible organizations:

1. **Simple**: one-word at a time is read from a memory bank
2. **Wide-memory**: N words at a time are read from a memory bank
3. **Interleaved**: K independent memory banks operate in parallel serving K requests simultaneously

Cache block transfer



Suppose a single main memory module

The *Memory* and *Cache* service time is t_M, t_{cache}, respectively

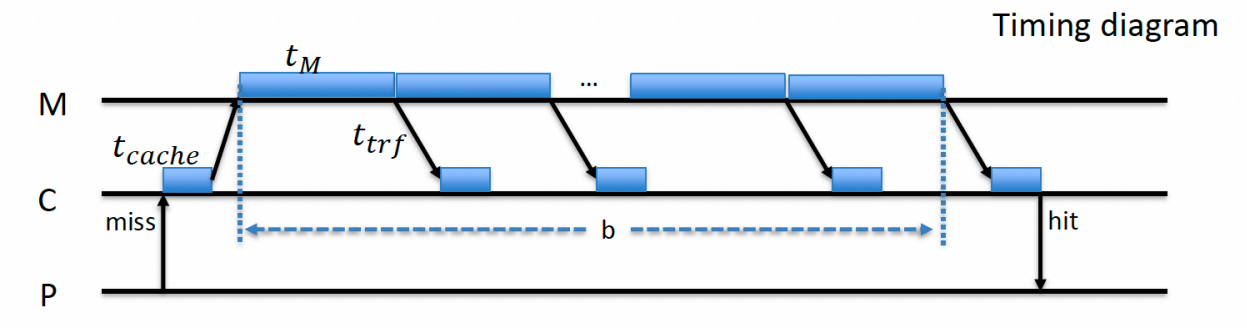
The offered memory bandwidth is $B_M = \frac{1}{t_M}$ and b=1

The memory access time as seen by the CPU is about

$$t_a = 2\,t_{trf} + 2\,t_{cache} + t_M$$

85

If the cache line holds *b>1* memory words, the cost to transfer the entire block in case of a cache miss is: $t_{miss} = 2t_{trf} + 2t_{cache} + bt_M \approx bt_M$



Cache issues

Caching is essential to reduce the von Neuman bottleneck and achieve performance on modern systems.

However, caching introduces some problems in ***multiprocessor/multicore systems***:

- *Cache coherence problem*
- False sharing:
  two unrelated variables are in the same cache block and
  accessed in read/write mode by different threads of the same process!